

Компьютерное зрение.

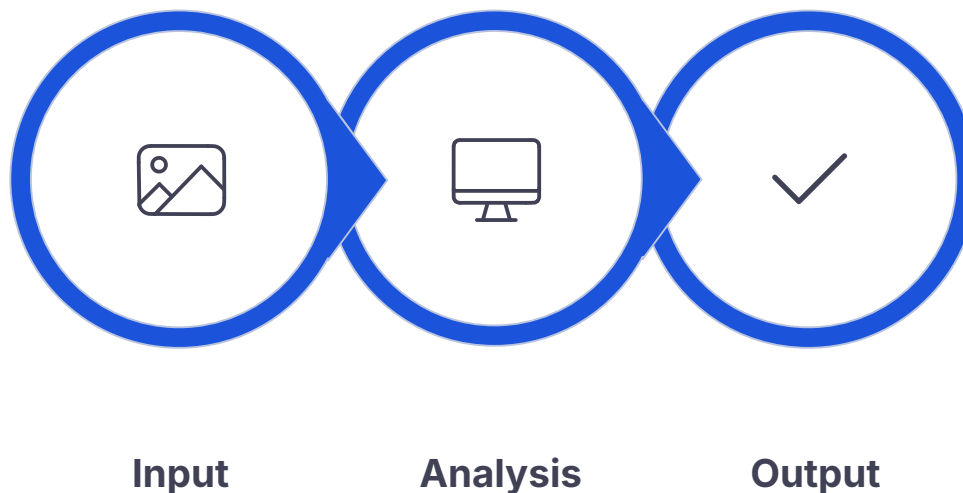
Основные задачи компьютерного и когнитивного зрения.

Перчихин О.И

Алексеевко Д.О.

Что такое компьютерное зрение?

Компьютерное зрение — это область искусственного интеллекта, которая позволяет компьютерам **видеть** и **интерпретировать** визуальные данные так же, как это делает человек.



В отличие от простой обработки изображений, которая фокусируется на изменении визуального сигнала, компьютерное зрение стремится к **глубокой смысловой интерпретации** увиденного. Оно распознает объекты, определяет их положение в пространстве, выделяет структуру сцены и анализирует действия.

С математической точки зрения, это переход от необработанных пиксельных данных к структурированному и осмысленному описанию наблюдаемой сцены.

Когнитивное зрение

Когнитивное зрение представляет собой более глубокий уровень визуального анализа, который выходит за рамки простого обнаружения и распознавания объектов.

В отличие от компьютерного зрения, которое фокусируется на вопросах **что и где находится на изображении**, когнитивное зрение стремится понять **что происходит и что это означает**, интерпретируя визуальные данные в широком контексте.



Этот подход включает моделирование контекста, причинно-следственных связей и намерений, интегрируя визуальную информацию с другими данными и знаниями о мире для достижения семантического понимания сцены.

Аппаратные основы компьютерного зрения

Эффективность систем компьютерного зрения напрямую зависит от качества и надежности данных, поступающих от аппаратной части.

Надежность данных

Алгоритмы компьютерного зрения не могут работать без надежных данных. Аппаратная часть, включающая сенсоры и камеры, является первичным источником информации о внешней среде.

Качество восприятия

Качество сенсоров напрямую определяет точность восприятия. Это, в свою очередь, влияет на способность системы принимать адекватные решения на основе полученной визуальной информации.

Влияние ограничений

Такие факторы, как низкая частота кадров, недостаточное разрешение или высокий уровень шума в изображениях, создают значительные ограничения, снижающие эффективность алгоритмов компьютерного зрения.

Аппаратное обеспечение

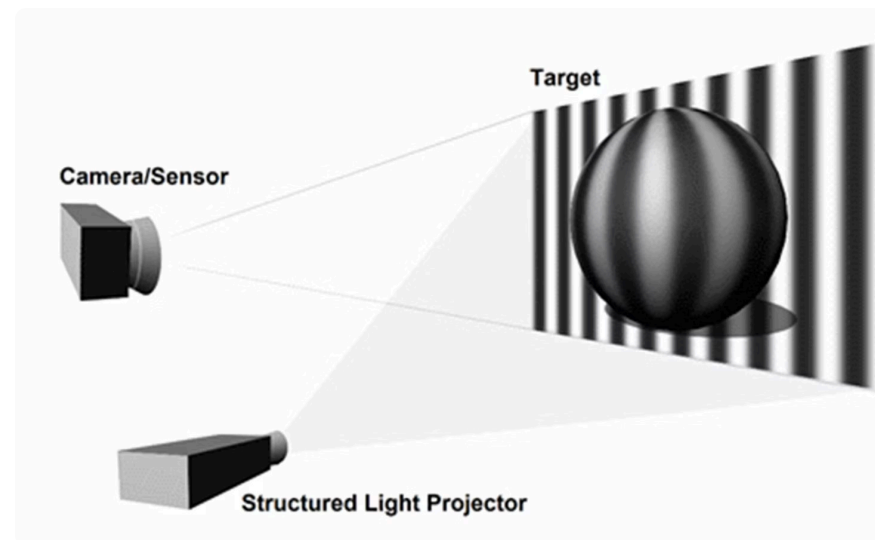
Аппаратный уровень играет критически важную роль в системах компьютерного зрения, поскольку он напрямую влияет на весь процесс обработки визуальных данных:

- **Формирует сырые данные:** Он является первичным источником визуальной информации, преобразуя физические сигналы в цифровой формат.
- **Определяет формат, частоту и качество данных:** Характеристики сенсоров (разрешение, частота кадров, глубина цвета) задают параметры входящего потока данных, что критично для последующего анализа.
- **Влияет на нагрузку вычислительной системы:** Объём и сложность потока данных напрямую коррелируют с требованиями к вычислительным ресурсам для их обработки.

Средства технического зрения

Средства технического зрения классифицируются по нескольким ключевым параметрам:

- **По принципу работы:**
 - Пассивные (например, обычные камеры, фиксирующие отраженный свет)
 - Активные (например, лидары или камеры с подсветкой, излучающие собственный свет)
- **По размерности:**
 - 2D сенсоры (традиционные камеры, захватывающие плоские изображения)
 - 3D сенсоры (например, датчики глубины, лидары, стереокамеры, предоставляющие информацию о расстоянии и объеме)
- **По способу взаимодействия:**
 - Контактные (требующие физического контакта с объектом для измерения)
 - Бесконтактные (большинство систем компьютерного зрения, работающие на расстоянии)

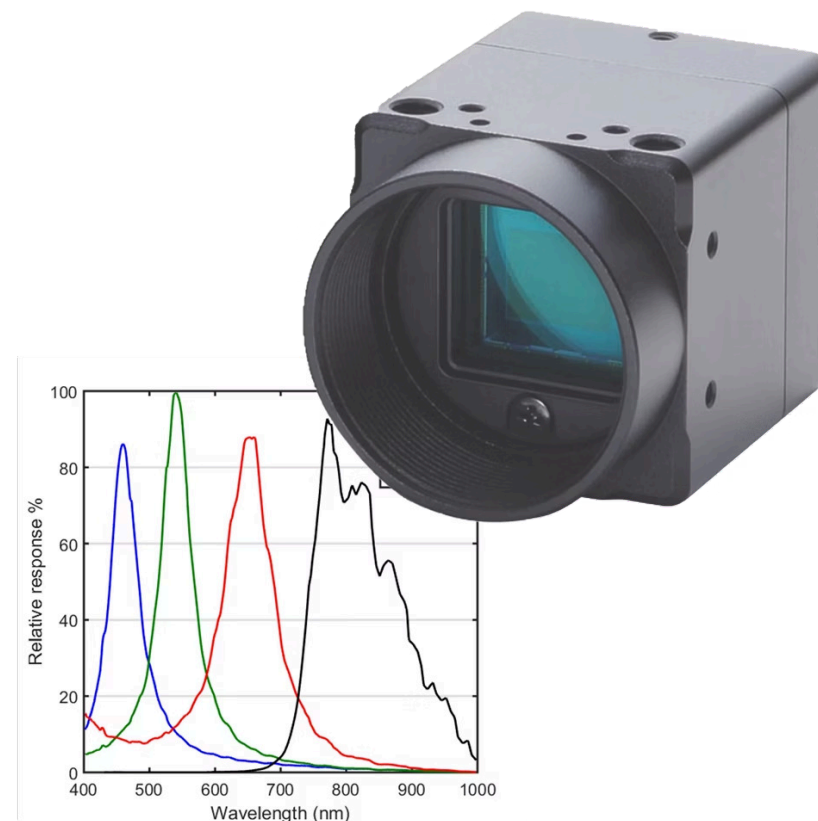


Пассивные сенсоры

Пассивные сенсоры не излучают собственный сигнал, а регистрируют отражённый или излучённый свет от объектов. Это делает их зависимыми от внешнего освещения, что является их основным ограничением в условиях низкой освещённости.

Они обычно просты в конструкции и экономичны, но их эффективность напрямую связана с доступностью света.

- **RGB камеры:** Захватывают цветное изображение.
- **Монохромные камеры:** Отличаются высокой чувствительностью и низким уровнем шума.
- **Тепловизоры:** Работают в инфракрасном диапазоне, регистрируя собственное тепловое излучение объектов.



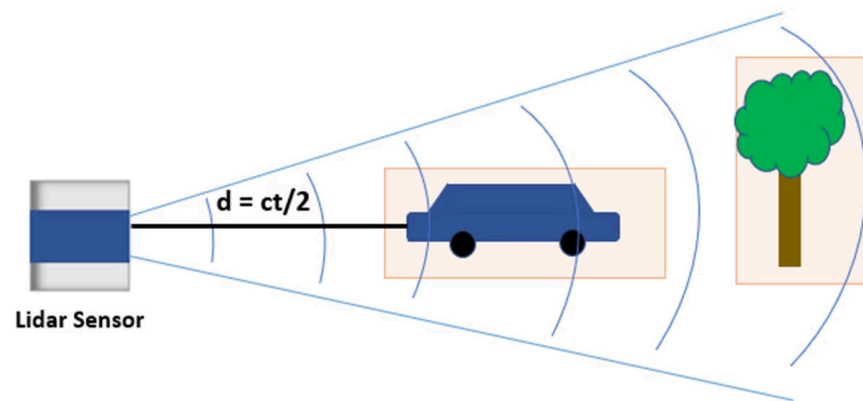
Активные сенсоры

В отличие от пассивных, активные сенсоры **излучают собственный сигнал** (свет, лазер, инфракрасное излучение) и затем регистрируют его отражение. Это позволяет им:

- Точно измерять глубину и расстояние до объектов.
- Эффективно работать в условиях слабого освещения или полной темноты, поскольку они не зависят от внешних источников света.
- Строить детальные 3D-модели окружения.

Основные примеры таких сенсоров включают:

- **Лидары (LiDAR):** Используют лазерное сканирование для измерения расстояний и создания облаков точек.
- **Time-of-Flight (ToF) камеры:** Измеряют время, за которое свет проходит путь от камеры до объекта и обратно.
- **Structured Light:** Проецируют известный паттерн света на сцену и анализируют его деформации для расчета глубины.
- **RGB-D камеры:** Комбинируют стандартное цветное изображение (RGB) с картой глубины (D) от активного сенсора.



Контактные и бесконтактные методы

Контактные методы:

Эти методы включают использование механических датчиков и щупов координатно-измерительных машин. Они обеспечивают **исключительно высокую точность** измерений, но при этом обладают **низкой скоростью** сбора данных. Контактные методы обычно применяются на этапе финального контроля геометрии, где критична каждая микронная деталь.

Бесконтактные методы:

Они используют камеры, лазеры и лидары для сбора данных, что исключает **риск повреждения** объекта. Главные преимущества — **высокая скорость** получения информации и возможность осуществления **онлайн-контроля** в реальном времени, что делает их незаменимыми для быстрых производственных циклов и динамичных сред.



Камеры и оптика

Камера является неотъемлемым элементом любой системы компьютерного зрения, служа основным источником визуальных данных. Её функциональность критична для преобразования физического света в цифровую информацию.

Основной источник данных

Камера служит глазами CV-системы, собирая первичную визуальную информацию из окружающего мира.

Преобразование сигнала

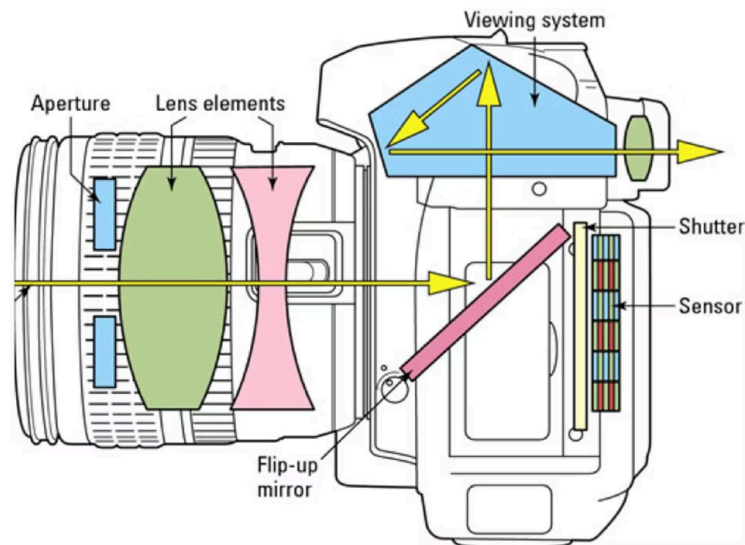
Она преобразует аналоговый свет в цифровые изображения, которые затем могут быть обработаны и проанализированы.

Влияние качества

Качество оптики и сенсора напрямую определяет точность и детализацию получаемых данных, влияя на весь анализ.

Баланс параметров

Оптимальное разрешение, частота кадров и чувствительность сенсора должны быть сбалансированы для конкретного применения.



Типы камер

Разнообразие камер позволяет адаптировать системы компьютерного зрения под специфические задачи и условия:

RGB-камеры

Захватывают цветное изображение в стандартном спектре видимого света, обеспечивая привычное визуальное представление для большинства повседневных приложений.

Камеры узкого диапазона спектра

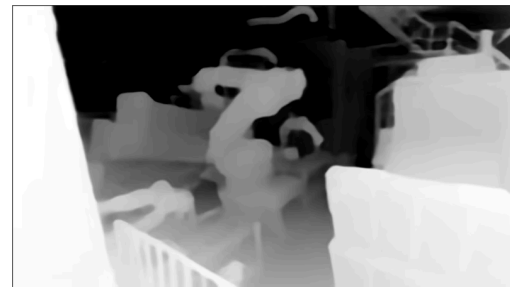
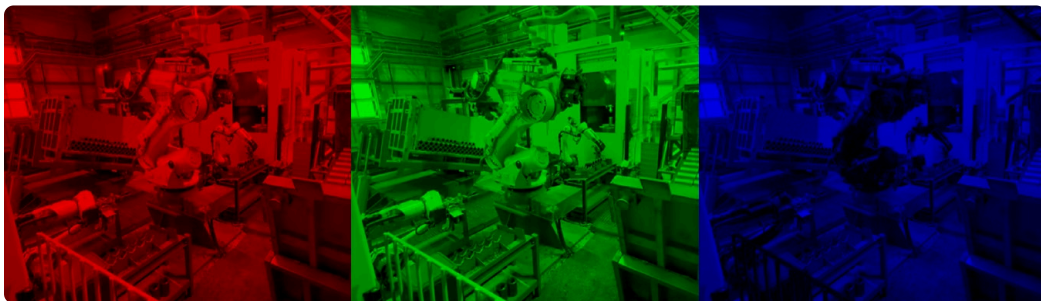
Фиксируют свет только в определённой узкой полосе длин волн. К ним относятся ИК и SWIR камеры, а также тепловизоры, что позволяет "видеть" за пределами человеческого восприятия, например, в темноте или сквозь туман.

Стереокамеры

Используют два оптических сенсора, расположенных на фиксированном расстоянии друг от друга, для получения трёхмерной информации о сцене, имитируя бинокулярное зрение.

RGB-D камеры

Комбинируют стандартное цветное изображение (RGB) с картой глубины (Depth) от активного сенсора, предоставляя одновременно информацию о цвете и расстоянии до каждого пикселя в сцене.



Промышленные камеры

Промышленные камеры разработаны для высокопроизводительных задач компьютерного зрения и работают в сложных условиях, значительно отличаясь от обычных камер своей конструкцией и функциональностью.

- **Высокая скорость съемки**

Способность фиксировать быстрые процессы и движения с высокой частотой кадров, критично для контроля производства и динамических систем.

- **Расширенный динамический диапазон**

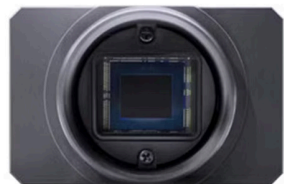
Эффективная работа в условиях контрастного освещения, сохраняя детали как в ярких, так и в темных участках изображения.

- **Устойчивость к условиям эксплуатации**

Конструктивная прочность для работы в условиях вибрации, экстремальных температур, пыли и влаги на промышленных объектах.

- **Узкая специализация**

Многие модели оптимизированы под конкретные задачи, например, машинное зрение, контроль качества, научные исследования или робототехника.



Аффинные преобразования

Аффинные преобразования — это фундаментальный класс геометрических преобразований, используемых в компьютерном зрении для манипуляции изображениями и 3D-моделями. Они сохраняют параллельность линий и прямые линии, но могут изменять расстояния, углы и площади. Это делает их мощным инструментом для калибровки камер, выравнивания изображений и моделирования движений объектов.

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & t_x \\ a_{21} & a_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

В этой матричной форме матрица 2x2 (элементы a_{11} , a_{12} , a_{21} , a_{22}) отвечает за **линейную часть** преобразования, включающую масштабирование, поворот и сдвиг (наклон). Вектор (t_x, t_y) представляет собой **вектор переноса**, сдвигающий объект по осям X и Y.

Смещение (Трансляция)

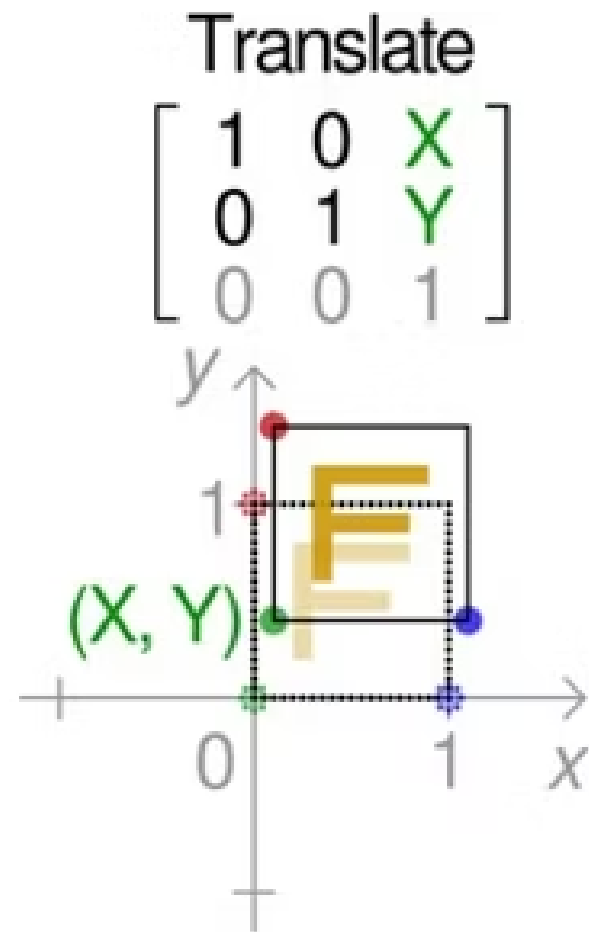
Смещение, или трансляция, — это аффинное преобразование, при котором каждый пиксель изображения перемещается на заданный вектор (t_x, t_y) . В двумерном пространстве новые координаты (x', y') рассчитываются по простым формулам:

$$x' = x + t_x, y' = y + t_y$$

Это преобразование сохраняет форму, размер и углы объекта, изменяя только его пространственное положение. Оно часто используется для позиционирования объектов, создания анимаций или компенсации движений камеры.

В матричной форме смещение выражается так:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$



Масштабирование (Scaling)

Масштабирование — это аффинное преобразование, изменяющее размер объекта. Оно позволяет увеличивать или уменьшать изображение, сохраняя при этом его пропорции ($s_x = s_y$, изотропное масштабирование) или растягивая/сжимая вдоль определённых осей ($s_x \neq s_y$, анизотропное масштабирование). Это ключевой инструмент для изменения размеров объектов, кадрирования и подгонки изображений.

Новые координаты (x', y') рассчитываются по формулам:

$$x' = s_x x$$

$$y' = s_y y$$

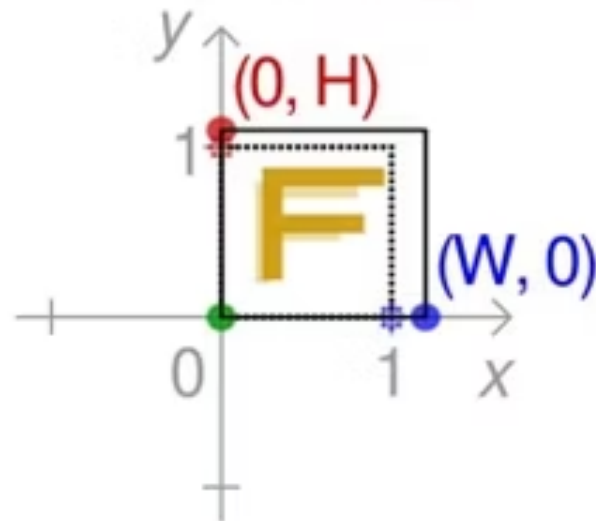
В матричной форме преобразование масштабирования выглядит так:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Здесь s_x и s_y — это коэффициенты масштабирования по осям X и Y. Когда $s_x = s_y$, масштабирование изотропное (пропорциональное). Если $s_x \neq s_y$, оно анизотропное (с растяжением или сжатием).

Scale about origin

$$\begin{bmatrix} W & 0 & 0 \\ 0 & H & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



Поворот (Rotation)

Поворот — это фундаментальное аффинное преобразование, которое вращает объект или изображение вокруг фиксированной точки, обычно начала координат $(0,0)$, на заданный угол θ . При повороте сохраняются форма и размер объекта, а также расстояния между точками и углы.

Новые координаты (x', y') каждой точки (x, y) после поворота вычисляются по следующим формулам:

$$x' = x \cos \theta - y \sin \theta$$

$$y' = x \sin \theta + y \cos \theta$$

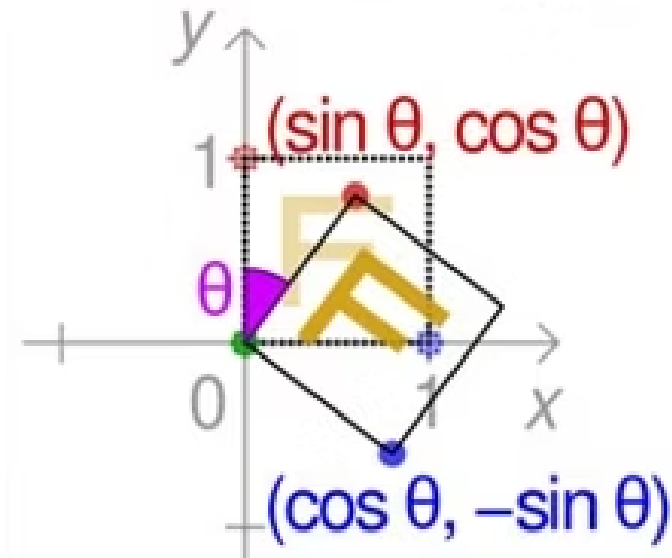
В матричной форме преобразование поворота выражается так:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Это преобразование широко используется для ориентации объектов в 3D-пространстве, выравнивания изображений и создания эффектов вращения в графике.

Rotate about origin

$$\begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



Сдвиг (Shear)

Сдвиг, или деформация сдвига, — это аффинное преобразование, которое наклоняет объект вдоль одной из осей, сохраняя при этом его параллельные линии, но изменяя углы.

Например, сдвиг по оси X:

$$x' = x + ky, y' = y$$

Здесь k — коэффициент сдвига, определяющий степень наклона. Это преобразование часто используется для создания эффектов перспективы, имитации искажений или коррекции геометрических ошибок.

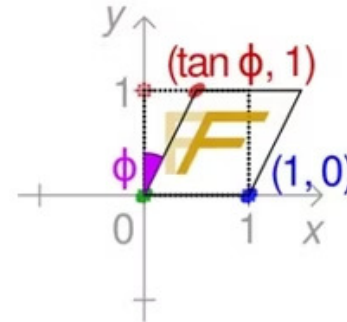
В матричной форме преобразование сдвига по оси X выглядит так:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & k & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

В результате сдвига параллельность линий сохраняется, но углы между ними изменяются, что придает объекту наклонный вид.

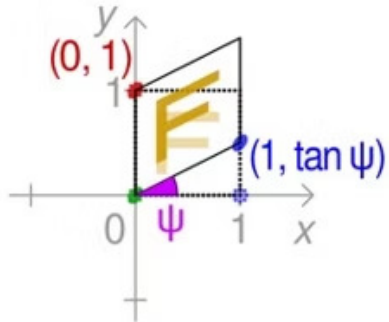
Shear in x direction

$$\begin{bmatrix} 1 & \tan \phi & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



Shear in y direction

$$\begin{bmatrix} 1 & 0 & 0 \\ \tan \psi & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



Отражение (Reflection)

Отражение, или зеркальное преобразование, создает зеркальную копию объекта относительно заданной оси или плоскости. Это аффинное преобразование сохраняет размер и форму объекта, но инвертирует его ориентацию, то есть превращает левую сторону в правую, а верхнюю в нижнюю (или наоборот, в зависимости от оси отражения). Отражения часто используются в компьютерной графике для создания симметричных изображений, эффектов зеркала или для корректировки ориентации объектов.

Например, отражение относительно оси X:

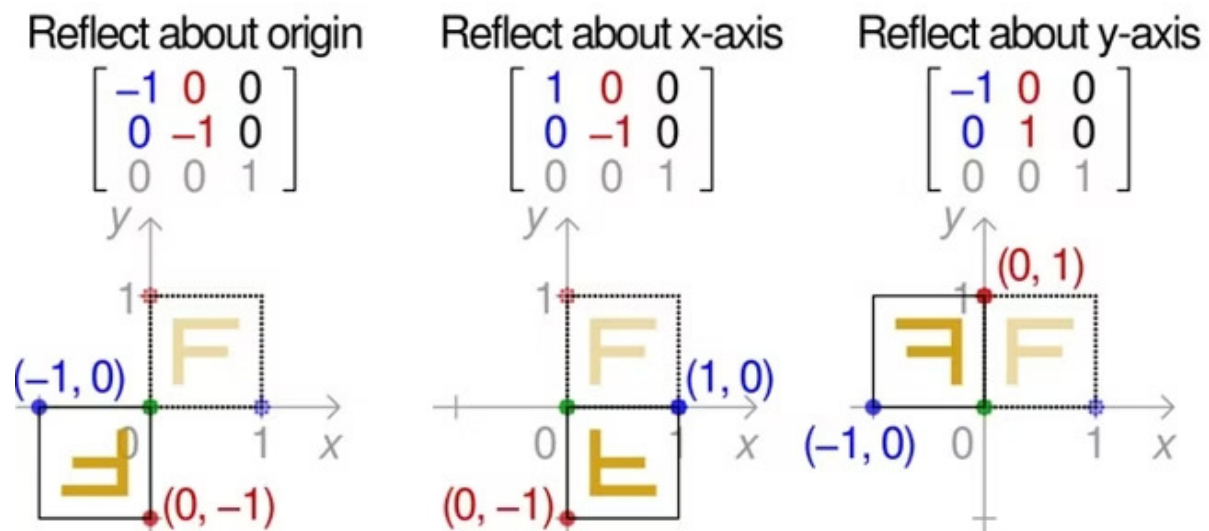
$$x' = x, y' = -y$$

В матричной форме преобразование отражения относительно оси X выглядит так:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Отражение (Reflection)

Аналогично можно выполнить отражение относительно оси Y или любой другой прямой.



Применение аффинных преобразований

Аффинные преобразования являются фундаментальным инструментом в компьютерном зрении, находя широкое применение в различных задачах обработки и анализа изображений. Они позволяют эффективно манипулировать геометрией изображений, сохраняя при этом ключевые свойства, такие как параллельность линий.



Аугментация данных

Используется для искусственного расширения обучающих наборов данных путём случайных сдвигов, масштабирований и поворотов, что повышает устойчивость и обобщающую способность моделей машинного обучения.



Выравнивание изображений

Применяется для коррекции положения и ориентации нескольких изображений, чтобы они точно совпадали друг с другом, что важно для создания панорам или совместного анализа данных.



Регистрация изображений

Помогает сопоставить изображения, полученные в разное время, с разных ракурсов или с помощью разных сенсоров, например, в медицинских приложениях или спутниковой съёмке.



Стабилизация видео

Позволяет устранить нежелательные дрожания и движения камеры в видеопотоке, делая его более плавным и приятным для просмотра или анализа.

Основные задачи компьютерного зрения

Компьютерное зрение охватывает широкий спектр задач, каждая из которых направлена на извлечение значимой информации из изображений и видео. Эти задачи формируют основу для множества приложений в различных отраслях.



Классификация изображений

Определение категории объекта или сцены на изображении (например, "автомобиль", "пейзаж").



Обнаружение объектов

Локализация и идентификация конкретных объектов в пределах изображения или видеопотока.



Распознавание лиц

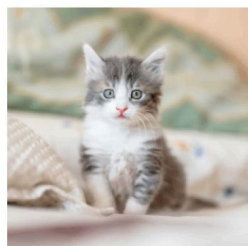
Идентификация человека по изображению его лица для аутентификации и персонализации.



Анализ эмоций и позы

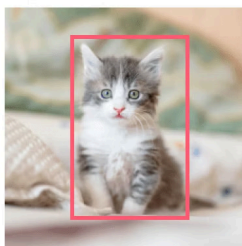
Определение эмоционального состояния человека и положения его тела в пространстве.

Classification



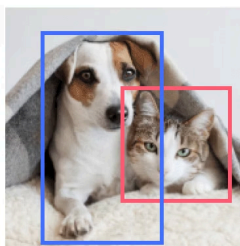
Cat

Classification + Localization



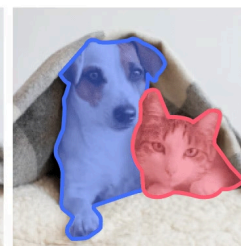
Cat

Object Detection



Cat, Dog

Instance Segmentation



Cat, Dog

Single object

Multiple objects

Классификация изображений

Классификация изображений — это фундаментальная задача компьютерного зрения, которая заключается в присвоении всему изображению одной или нескольких меток класса. Это означает, что система анализирует визуальные данные и определяет, к какой категории они принадлежат.

Формально это можно представить как функцию:

$$f : R^{H \times W \times C} \rightarrow \{1, \dots, K\}$$

где $H \times W \times C$ — это высота, ширина и количество каналов входного изображения, а K — количество возможных классов. Модель преобразует тензор пикселей в одну семантическую метку, такую как «кошка», «автомобиль», «дефект» или «норма».

В классическом подходе предполагается, что на изображении присутствует один доминирующий объект, и результатом является одна единственная метка.

Классификация изображений

В процессе классификации изображений, современные модели компьютерного зрения не просто выдают одну метку класса, а рассчитывают **распределение вероятностей** для всех возможных категорий. Это позволяет оценить уверенность модели в своем предсказании.

$$p(y \mid I)$$

На основании этого распределения, выбирается класс с максимальной вероятностью как окончательное предсказание:

$$\hat{y} = \arg \max_k p(y = k \mid I)$$

Такой подход обеспечивает гибкость, позволяет учитывать неопределенность и является основой для использования таких функций потерь, как кросс-энтропия, что улучшает процесс обучения модели.

Основные типы классификации



Бинарная классификация

Разделение изображений на два взаимоисключающих класса (например, "дефект" / "норма", "кот" / "не кот").



Мультиклассовая классификация

Присвоение изображению одной из K возможных категорий, где каждый объект принадлежит строго к одному классу (например, "кошка", "собака", "птица").



Мульти-лейбл классификация

Изображение может одновременно принадлежать нескольким классам (например, на одном изображении могут быть "кошка", "собака" и "трава").

Детекция объектов

Детекция объектов — это задача компьютерного зрения, которая включает не только определение присутствия объектов на изображении, но и точное указание их местоположения. В отличие от простой классификации, детекция предоставляет информацию о том, где именно находится каждый объект и какова его категория.

Для каждого найденного объекта модель обычно выдает следующую информацию:



Координаты рамки

Прямоугольник, ограничивающий объект, часто представленный как $(x_{min}, y_{min}, x_{max}, y_{max})$.



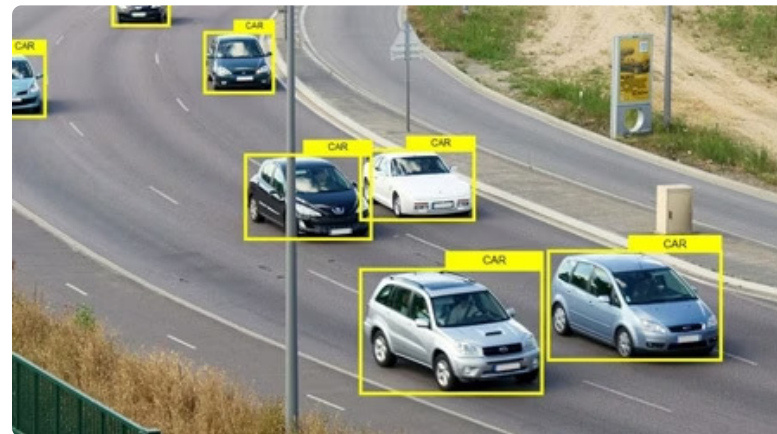
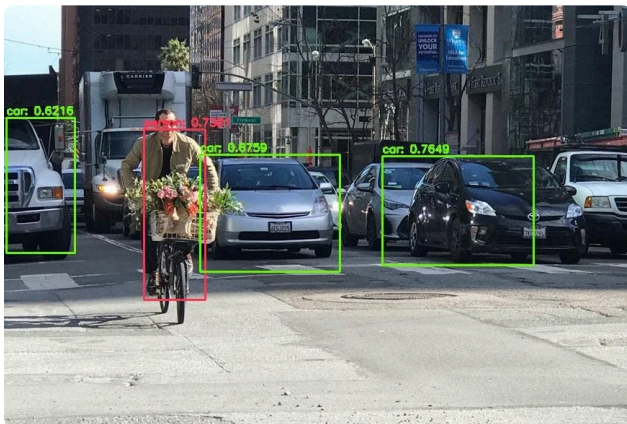
Метка класса

Название категории объекта, например, "автомобиль", "человек" или "велосипед".



Доверительная вероятность

Числовое значение, отражающее уверенность модели в правильности своего предсказания (от 0 до 1).



Параметризация ограничивающих рамок

Ограничивающие рамки (**bounding boxes**), используемые для точной локализации объектов на изображении, могут быть представлены несколькими способами. Выбор формы записи зависит от конкретной задачи и особенностей алгоритма, но две наиболее распространенные параметризации:

Координатная форма

Эта форма описывает рамку с использованием координат верхнего левого и нижнего правого углов.

$$(x_{min}, y_{min}, x_{max}, y_{max})$$

Она интуитивно понятна и удобна для визуализации, а также для расчета площади пересечения (IoU) при оценке качества детекции.

Центр + размеры

В этой форме рамка определяется координатами своего центра и её габаритами (шириной и высотой).

$$(x_c, y_c, w, h)$$

Эта параметризация часто используется в задачах регрессии параметров рамки в моделях глубокого обучения, так как позволяет более явно управлять положением и размером объекта.

Детекция объектов: Сочетание классификации и регрессии

Задача детекции объектов является более сложной по сравнению с простой классификацией, так как она не только определяет категорию объекта, но и его точное местоположение на изображении. По сути, детекция объектов объединяет две ключевые задачи компьютерного зрения:



Классификация

Определение **категории объекта** (например, "человек", "автомобиль", "собака"). Модель присваивает каждому обнаруженному региону изображения метку класса.



Регрессия

Предсказание **координат ограничивающей рамки** (bounding box) для каждого объекта. Это позволяет точно локализовать объект в пространстве изображения.

Особенности задачи детекции объектов

Детекция объектов сопряжена с рядом уникальных вызовов, которые требуют специализированных алгоритмов и архитектур моделей.



Неизвестное число объектов

На одном изображении может быть от нуля до нескольких десятков или сотен объектов. Модель должна уметь адаптироваться к этому.



Перекрытие объектов

Объекты часто перекрываются или находятся очень близко друг к другу, что затрудняет их индивидуальную локализацию и классификацию.



Масштаб и разнообразие форм

Объекты могут иметь сильно различающийся размер и соотношение сторон, а также представать в разных ракурсах и условиях освещения.



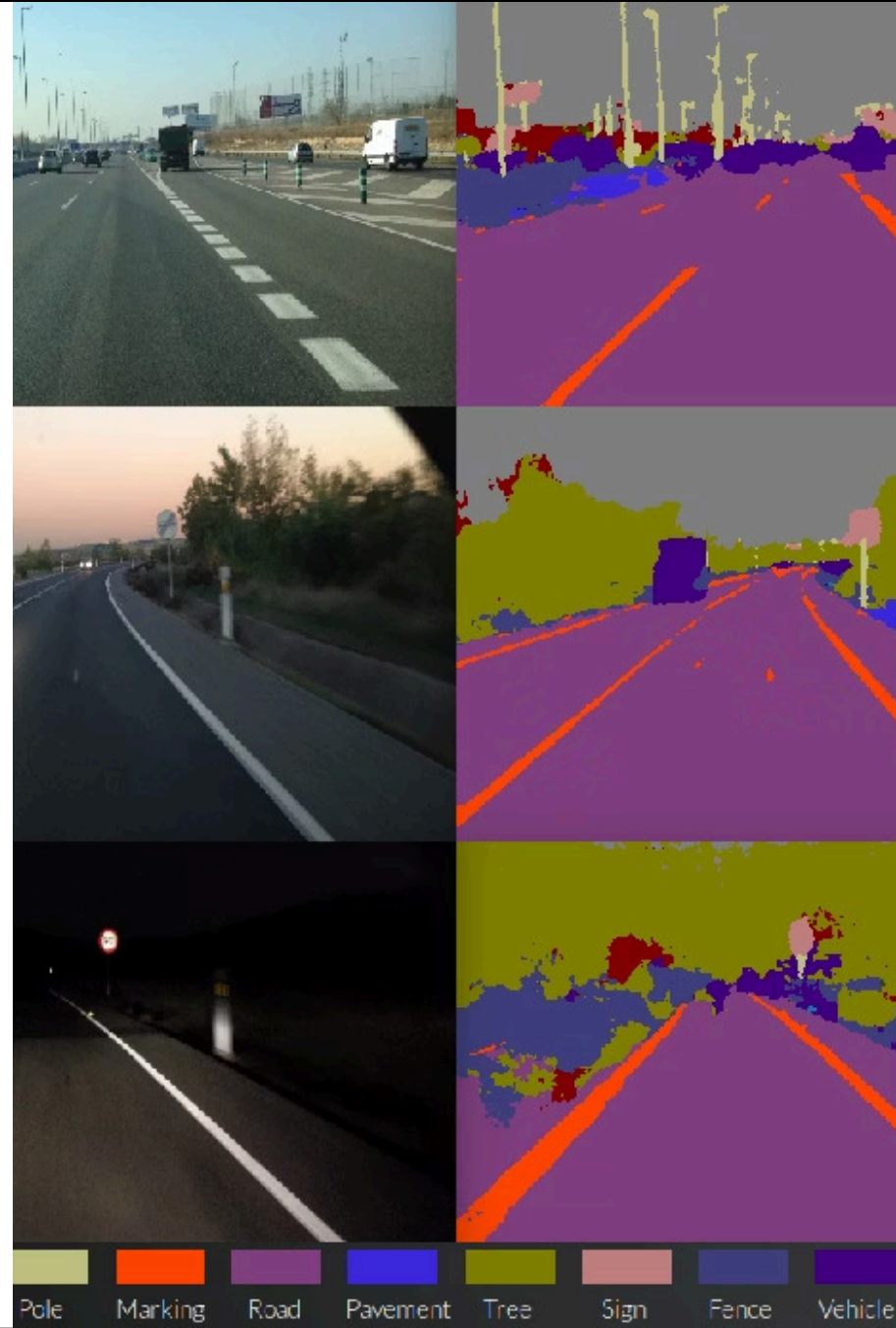
Подавление дублирующих предсказаний

Модель часто генерирует несколько пересекающихся рамок для одного и того же объекта. Необходимо использовать методы, такие как Non-Maximum Suppression (NMS), для выбора наиболее точной рамки.

Сегментация изображений

Сегментация изображений — это задача компьютерного зрения, которая заключается в разбиении изображения на смысловые области или объекты на пиксельном уровне. В отличие от детекции объектов, которая просто очерчивает объекты ограничивающими рамками, сегментация присваивает каждому пикселю изображения метку определенного класса.

Это позволяет получить гораздо более точное представление о форме и границах каждого объекта, что критически важно для многих приложений, от автономного вождения до медицинской диагностики, где требуется высокая точность определения контуров.



Особенности задачи сегментации

Сегментация изображений — это более сложная задача по сравнению с классификацией и детекцией объектов, поскольку она имеет следующие уникальные особенности и вызовы:



Плотное предсказание

Требуется классифицировать каждый отдельный пиксель изображения, присваивая ему метку класса. Это значительно увеличивает объем вычислений и сложность модели.



Чувствительность к границам

Модели сегментации должны быть очень точными при определении границ объектов, чтобы избежать ошибок в критически важных областях, таких как медицинская диагностика или автономное вождение.



Высокие вычислительные требования

Из-за необходимости обработки информации на уровне каждого пикселя, задачи сегментации требуют значительно больших вычислительных ресурсов и мощных графических процессоров.



Точная разметка данных

Для обучения моделей сегментации необходимы очень точные аннотации, где каждый пиксель размечен вручную. Это трудоемкий и дорогостоящий процесс.

Обзор библиотек для работы с изображениями

Работа с изображениями в задачах компьютерного зрения требует инструментов для загрузки, преобразования, анализа и визуализации данных. В экосистеме Python сложился устойчивый набор библиотек, которые покрывают разные уровни работы — от низкоуровневой обработки пикселей до интеграции с нейросетевыми моделями.

Эти библиотеки предоставляют мощные функциональные возможности, упрощающие разработку и внедрение сложных алгоритмов обработки изображений и машинного обучения.



NumPy

NumPy является краеугольным камнем большинства библиотек компьютерного зрения в Python. Он предоставляет мощную основу для работы с числовыми данными, которая критически важна для обработки изображений.



Представление изображений

В Python изображение обычно представляется как **многомерный массив NumPy** размера $H \times W \times C$, где H — высота, W — ширина, а C — количество каналов (например, RGB).



Эффективные операции

NumPy обеспечивает высокоэффективные операции над этими массивами: **арифметические вычисления**, **индексация**, **трансформации** и гибкую работу с различными типами данных пикселей.



Фундамент CV

Практически все высокоуровневые операции с изображениями в таких библиотеках, как OpenCV, scikit-image или Pillow, в конечном итоге сводятся к **манипуляциям с массивами NumPy**.

NumPy

Базовые команды для работы с массивами

****Создание массивов:****

- * `np.arange()` - создание массива с равномерно распределенными значениями
- * `np.linspace()` - создание массива с заданным количеством элементов
- * `np.zeros()` - создание массива, заполненного нулями
- * `np.eye()` - создание единичной матрицы
- * `np.matrix()` - создание матрицы

Математические операции

****Арифметические операции:****

- * `+` - сложение массивов
- * `-` - вычитание массивов
- * `*` - поэлементное умножение
- * `@` - матричное умножение
- * `**` - возведение в степень

****Статистические функции:****

- * `np.sum()` - сумма элементов
- * `np.mean()` - среднее значение
- * `np.median()` - медиана
- * `np.std()` - стандартное отклонение
- * `np.var()` - дисперсия
- * `np.min()`/`np.max()` - минимум/максимум

Преобразование массивов

- * `np.ravel()` - преобразование матрицы в одномерный массив
- * `np.transpose()` - транспонирование матрицы
- * `np.reshape()` - изменение формы массива
- * `np.swapaxes()` - перестановка осей массива

Работа с условиями

- * `np.where()` - условная обработка элементов
- * `np.logical_and()`/`np.logical_or()` - логические операции
- * `np.greater()`/`np.less()` - сравнение массивов

Генерация данных

- * `np.random.rand()` - случайные числа от 0 до 1
- * `np.random.randint()` - случайные целые числа
- * `np.random.permutation()` - перемешивание элементов

Изучение документации библиотеки **numpy** остаётся на **самостоятельное изучение**

OpenCV

OpenCV (Open Source Computer Vision Library) — одна из наиболее распространённых библиотек для компьютерного зрения. Она предоставляет мощные, оптимизированные инструменты для широкого спектра задач, от обработки изображений до машинного обучения.

Ключевые возможности

- Загрузка и сохранение изображений и видео различных форматов.
- Геометрические преобразования (изменение размера, обрезка, поворот) и аффинные трансформации.
- Фильтрация изображений (размытие, повышение резкости) и свёртки для выделения признаков.
- Выделение признаков (например, SIFT, SURF, ORB) и сопоставление объектов.
- Работа с камерой, включая захват видеопотоков в реальном времени.
- Базовые алгоритмы детекции (например, детекция лиц с помощью каскадов Хаара) и трекинга объектов.

OpenCV ориентирована на высокую производительность благодаря оптимизированным алгоритмам и C++ ядру, что позволяет использовать её как в исследовательских проектах, так и в промышленных системах реального времени.

Pillow (PIL)

Базовая обработка изображений

Pillow — это мощная, но простая в использовании библиотека, предназначенная для базовых операций с изображениями.

Основные функции

Включает функции для чтения, записи, изменения размера и преобразования цветовых пространств изображений.

Предварительная обработка

Чаще всего используется для предварительной обработки изображений или подготовки данных перед подачей их в более сложные алгоритмы компьютерного зрения.

Простота использования

По своей структуре Pillow проще OpenCV и идеально подходит для задач, не требующих высокопроизводительной обработки в реальном времени.

PyTorch / TensorFlow

PyTorch и TensorFlow — это ведущие библиотеки для глубокого обучения, которые также играют ключевую роль в компьютерном зрении. Они предоставляют комплексные инструменты для создания и обучения современных моделей машинного зрения.



Фокус на глубоком обучении

Эти библиотеки являются основной платформой для разработки и внедрения сложных нейросетевых архитектур.



Модули для изображений

Включают функции для тензорных операций, аугментации и загрузки данных, что крайне важно для предобработки изображений.



Стандарт для моделей CV

В современных системах компьютерного зрения они стали отраслевым стандартом для построения и обучения высокопроизводительных моделей.

Обзор возможностей библиотеки OpenCV.

Детекция номерных знаков без ML

Базовые операции OpenCV (cv2)

****Загрузка и сохранение:****

- * `cv2.imread()` – загрузка изображения
- * `cv2.imwrite()` – сохранение изображения
- * `cv2.VideoCapture()` – захват видео/камеры
- * `cap.read()` – чтение кадра
- * `cap.release()` – освобождение камеры

****Отображение:****

- * `cv2.imshow()` – показать изображение
- * `cv2.waitKey()` – ожидание нажатия клавиши
- * `cv2.destroyAllWindows()` – закрытие окон

Цветовые преобразования

- * `cv2.cvtColor()` – преобразование цветового пространства
- * `cv2.COLOR\_BGR2GRAY` – BGR → Grayscale
- * `cv2.COLOR\_BGR2RGB` – BGR → RGB
- * `cv2.COLOR\_BGR2HSV` – BGR → HSV

Геометрические операции

- * `cv2.resize()` – изменение размера изображения
- * `cv2.getRotationMatrix2D()` – матрица поворота
- * `cv2.warpAffine()` – аффинное преобразование
- * `cv2.getPerspectiveTransform()` – вычисление матрицы гомографии
- * `cv2.warpPerspective()` – перспективное преобразование
- * `img[y1:y2, x1:x2]` – обрезка (crop)

Размытие и фильтрация

- * `cv2.GaussianBlur()` – гауссово размытие
- * `cv2.medianBlur()` – медианное размытие
- * `cv2.bilateralFilter()` – билинейная фильтрация
- * `cv2.filter2D()` – произвольная свёртка

Пороговая обработка (бинаризация)

- * `cv2.threshold()` – глобальная бинаризация
- * `cv2.THRESH\_BINARY` – бинарный порог
- * `cv2.THRESH\_OTSU` – метод Отсу
- * `cv2.adaptiveThreshold()` – адаптивная бинаризация

Морфологические операции

- * `cv2.getStructuringElement()` – создание ядра
- * `cv2.erode()` – эрозия
- * `cv2.dilate()` – дилатация
- * `cv2.morphologyEx()` – расширенные операции
- * `cv2.MORPH\_OPEN` – открытие
- * `cv2.MORPH\_CLOSE` – закрытие
- * `cv2.MORPH\_TOPHAT` – Top Hat
- * `cv2.MORPH\_BLACKHAT` – Black Hat

Базовые операции OpenCV (cv2)

Поиск границ и контуров

- * `cv2.Canny()` – детектор границ
- * `cv2.findContours()` – поиск контуров
- * `cv2.drawContours()` – отрисовка контуров
- * `cv2.boundingRect()` – ограничивающий прямоугольник
- * `cv2.minAreaRect()` – минимальный повернутый прямоугольник
- * `cv2.contourArea()` – площадь контура
- * `cv2.arcLength()` – периметр контура
- * `cv2.approxPolyDP()` – аппроксимация контура

Логические операции (маски)

- * `cv2.bitwise\_and()` – логическое AND
- * `cv2.bitwise\_or()` – логическое OR
- * `cv2.bitwise\_xor()` – XOR
- * `cv2.bitwise\_not()` – инверсия
- * `mask=` – применение маски

Преобразование Хафа

- * `cv2.HoughLines()` – поиск линий
- * `cv2.HoughLinesP()` – вероятностный Хаф
- * `cv2.HoughCircles()` – поиск окружностей

Выделение признаков

- * `cv2.ORB\_create()` – ORB признаки
- * `orb.detectAndCompute()` – ключевые точки и дескрипторы
- * `cv2.BFMatcher()` – сопоставление дескрипторов

Рисование примитивов

- * `cv2.rectangle()` – прямоугольник
- * `cv2.circle()` – окружность
- * `cv2.line()` – линия
- * `cv2.putText()` – текст на изображении

Представление цветного изображения

Цветное изображение в цифровом формате представляется как трёхмерный массив пикселей, который можно описать размерами:

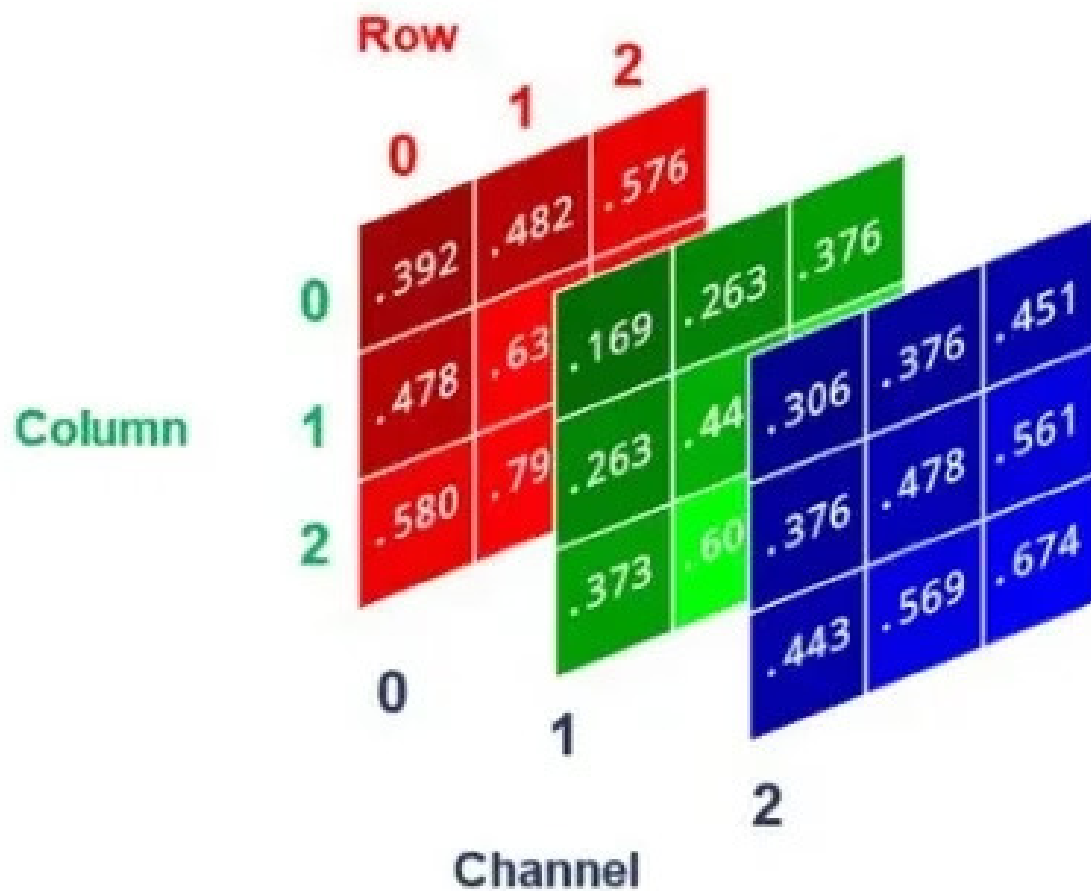
- **H** — Высота (количество строк пикселей)
- **W** — Ширина (количество столбцов)
- **C** — Число цветовых каналов

Для стандартного цветного изображения используется три канала ($C = 3$): **R** (Красный), **G** (Зелёный) и **B** (Синий). Каждый пиксель изображения — это тройка чисел (R, G, B) , значения которых обычно находятся в диапазоне от 0 до 255.

Например:

- $(255, 0, 0)$ — чистый красный
- $(0, 255, 0)$ — чистый зелёный
- $(0, 0, 255)$ — чистый синий
- $(0, 0, 0)$ — чёрный
- $(255, 255, 255)$ — белый

Представление цветного изображения



Цветовые преобразования



Здесь показано **преобразование цветного изображения в градации серого**.

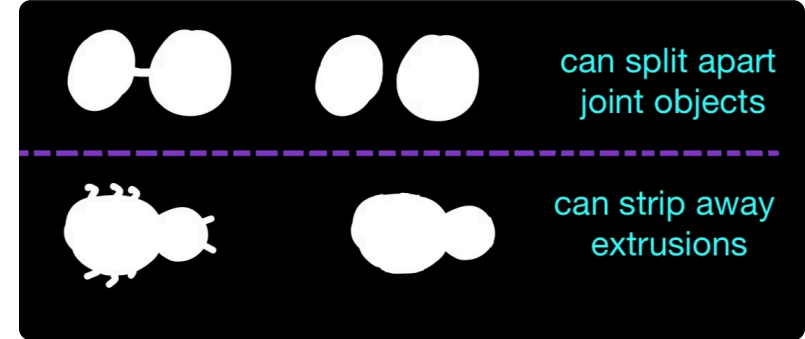
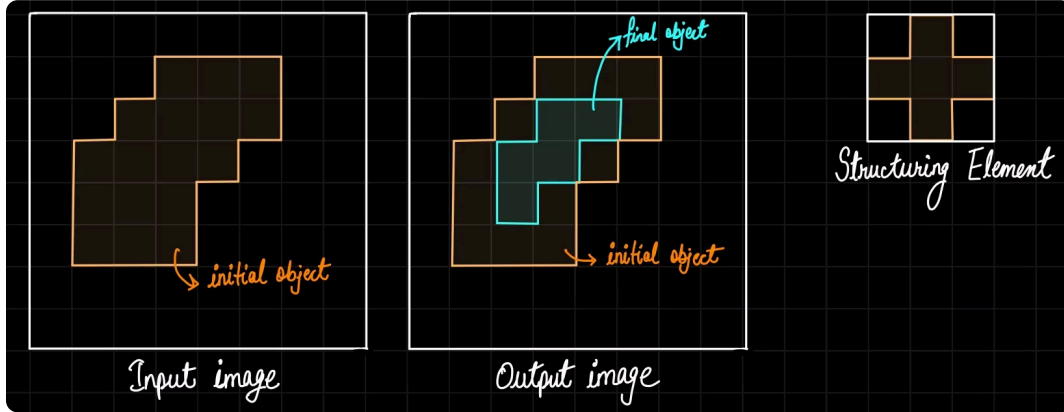
- из трёхканального RGB-изображения формируется одноканальное изображение яркости;
- цветовая информация удаляется;
- остаётся только интенсивность света (luminance);
- упрощается структура данных: $H \times W \times 3 \rightarrow H \times W$
- Математически это взвешенная сумма: $Y=0.299R+0.587G+0.114B$

Морфологические операции



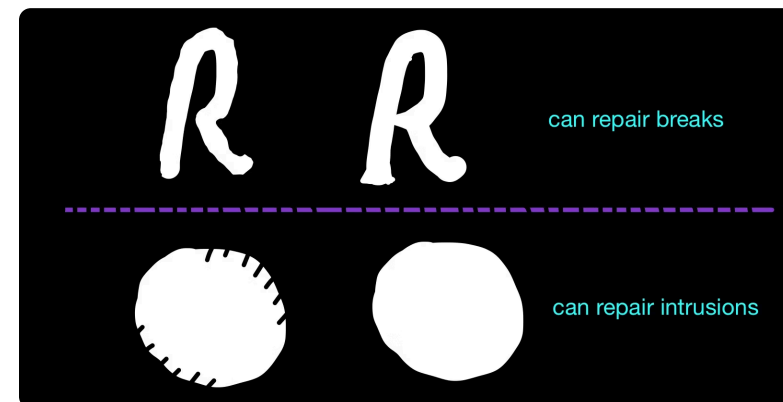
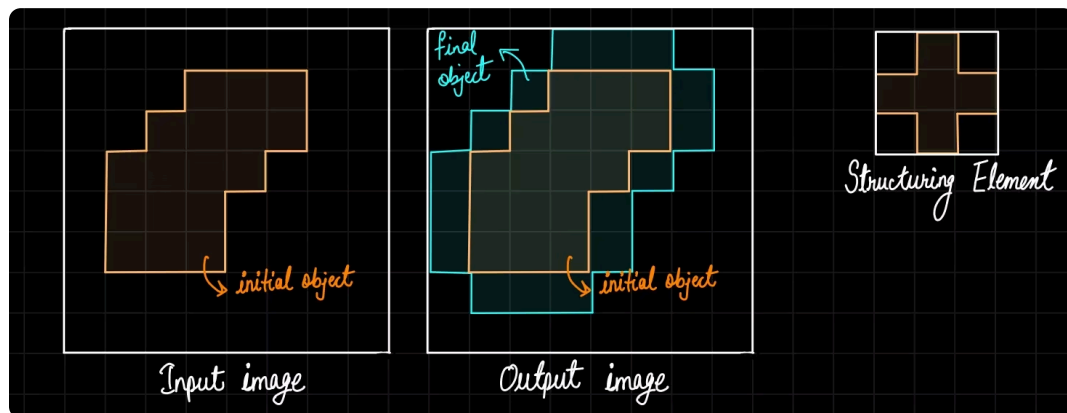
Морфологические операции определены только для бинарных изображений (все пиксели либо 0, либо 255)

Erode(Эрозия)



Эрозия — это базовая морфологическая операция, применяемая к бинарным или градационным изображениям, которая приводит к «сужению» объектов за счёт удаления пикселей по их границе.

Dilate(расширение)



Расширение(дилатация) - это морфологическая операция, противоположная эрозии, которая приводит к **расширению** объектов за счёт добавления пикселей по их границе.

Opening и Closing

Opening (Открытие)

Определение:

$$(A \circ B)$$

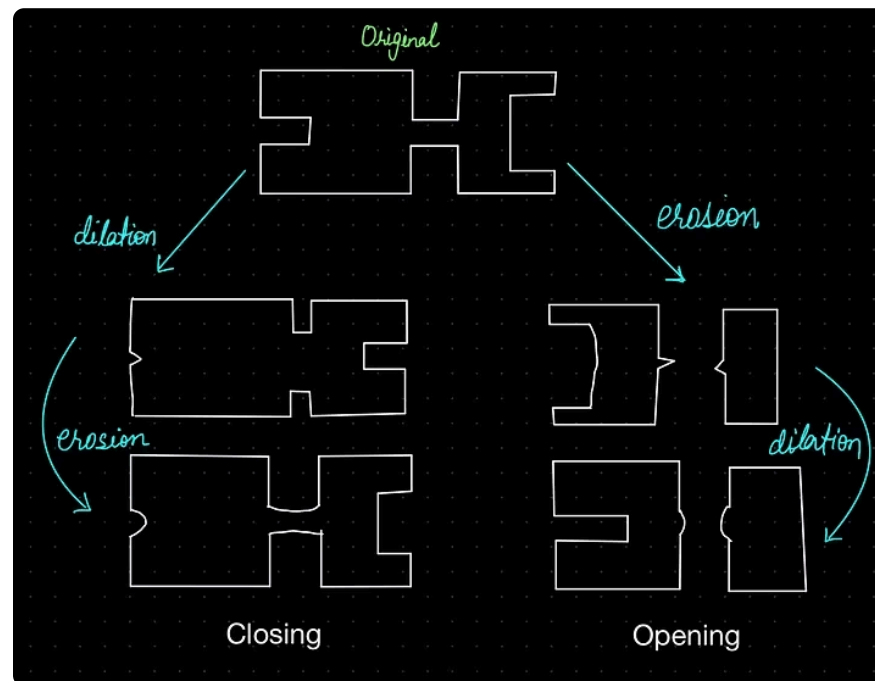
Opening — это эрозия, за которой следует дилатация с тем же структурным элементом

Closing (Заккрытие)

Определение:

$$(A \bullet B)$$

Closing — это дилатация, за которой следует эрозия



Opening и Closing

Открытие (Opening)

Последовательность **эрозии**, затем **дилатации** с одним и тем же структурным элементом. Эта операция эффективно **удаляет мелкие объекты и шумы** (например, соринки), а также **сглаживает контуры** и разрывает тонкие соединения между объектами.

Обе операции сохраняют общую форму крупных объектов и часто применяются последовательно для комплексной очистки изображения перед сегментацией или анализом.

Заккрытие (Closing)

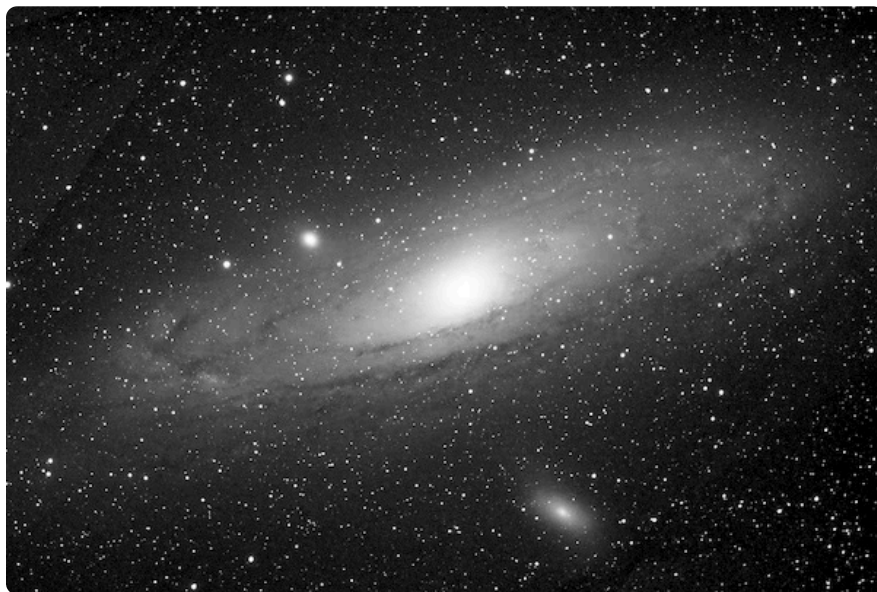
Последовательность **дилатации**, затем **эрозии** с тем же структурным элементом. Заккрытие **заполняет мелкие отверстия и разрывы** внутри объектов, **сглаживает вогнутости** контуров и объединяет близко расположенные части объекта, закрывая узкие промежутки.

Top hat

Top-Hat — это операция, предназначенная для выделения мелких ярких или тёмных структур на фоне более крупных объектов или неравномерного освещения. Она используется для усиления локальных деталей и подавления медленно меняющегося фона.

White Top-Hat

White Top-Hat определяется как разность исходного изображения и его открытия



Top hat

Top-Hat — это операция, предназначенная для выделения мелких ярких или тёмных структур на фоне более крупных объектов или неравномерного освещения. Она используется для усиления локальных деталей и подавления медленно меняющегося фона.

Black Top-Hat

Black Top-Hat определяется как разность между результатом closing и исходным изображением



Сравнение White Top-Hat и Black Top-Hat

White Top-Hat

Определение: $A - (A \circ B)$

- Выделяет мелкие светлые детали, удаленные операцией opening.
- Находит мелкие светлые объекты на темном фоне (пылинки, блики, царапины).
- Используется для обнаружения дефектов и выравнивания освещения.
- Усиливает контраст мелких светлых деталей для сегментации.

Пример: Выделение мелких капель припоя или бликов на печатной плате.

Black Top-Hat

Определение: $(A \bullet B) - A$

- Выделяет мелкие темные детали, заполненные операцией closing.
- Находит мелкие темные объекты на светлом фоне (трещины, поры, пятна).
- Применяется для обнаружения дефектов: трещин, сколов в материалах.
- Подчеркивает тени и затемнения, полезен в медицинской визуализации.

Пример: Подчеркивание тонких темных трещин на снимке горной породы.

Применение black hat



- подавляется крупный светлый фон;
- усиливаются мелкие тёмные детали;
- подчёркиваются тёмные элементы на светлом фоне (номер, решётка радиатора, текстура кузова, границы деталей);
- изображение становится контурным с акцентом на структурные элементы.

Фильтр Гаусса и Собеля

Размытие по Гауссу (Gaussian Blur)

Ядро Гаусса — это симметричная матрица, где веса пикселей уменьшаются по мере удаления от центра.

Пример ядра 3×3 (при $\sigma \approx 0,8$):

```
1/16 * [  
  [1, 2, 1],  
  [2, 4, 2],  
  [1, 2, 1]  
]
```

Пример ядра 5×5 (при $\sigma \approx 1$):

```
1/273 * [  
  [1, 4, 7, 4, 1],  
  [4, 16, 26, 16, 4],  
  [7, 26, 41, 26, 7],  
  [4, 16, 26, 16, 4],  
  [1, 4, 7, 4, 1]  
]
```

Коэффициент перед матрицей используется для нормализации, сохраняя общую яркость. Параметры σ и размер ядра влияют на степень размытия.

Фильтр Собеля (Sobel Filter)

Фильтр Собеля использует два ядра 3×3 для вычисления градиента яркости, выделяя границы по горизонтали и вертикали.

Ядро для горизонтальных границ (G_x , вертикальные края):

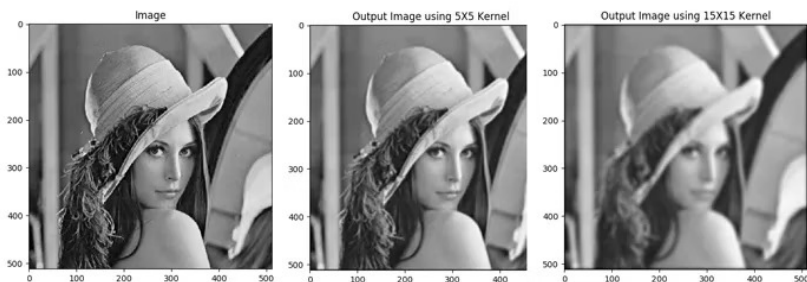
```
[  
  [-1, 0, 1],  
  [-2, 0, 2],  
  [-1, 0, 1]  
]
```

Ядро для вертикальных границ (G_y , горизонтальные края):

```
[  
  [-1, -2, -1],  
  [0, 0, 0],  
  [1, 2, 1]  
]
```

Эти ядра позволяют обнаружить резкие изменения интенсивности пикселей, что соответствует границам объектов на изображении.

Размытие по Гауссу (Gaussian Blur)



Размытие по Гауссу — это результат свёртки изображения с гауссовым ядром, широко используемый для уменьшения шума и сглаживания деталей.

Эффект:

- Подавляет высокочастотные компоненты (резкие границы, шум).
- Сглаживает детали и уменьшает контраст мелких структур.
- Степень размытия зависит от размера ядра: чем оно больше, тем сильнее эффект.

Математически это **свёртка**:

$$I'(x, y) = \sum_{u, v} I(x - u, y - v) K(u, v)$$

где $K(u, v)$ — гауссово ядро, значения которого распределены по нормальному закону:

$$K(u, v) \propto \exp\left(-\frac{u^2 + v^2}{2\sigma^2}\right)$$

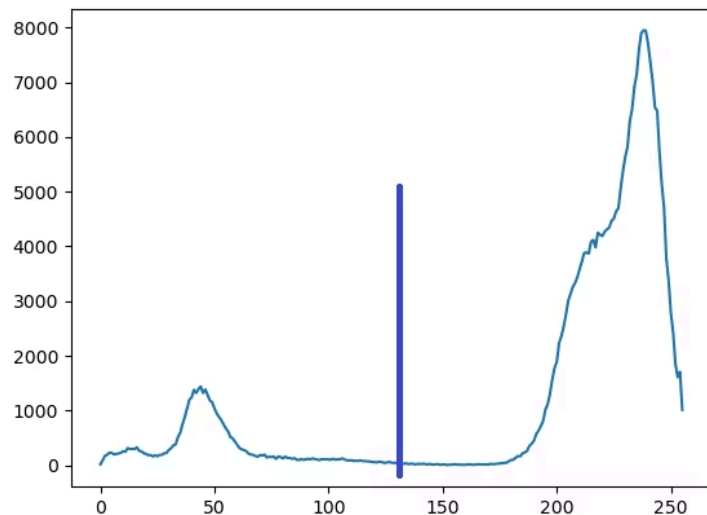
Центральные пиксели получают больший вес, удалённые — меньший.

Gaussian blur



- устраняет шум,
- используется перед детекцией границ
- применяется для подавления мелких артефактов.

Otsu-бинаризация



Otsu-бинаризация — это автоматический метод преобразования полутонового изображения в бинарное путем выбора оптимального глобального порога t .

- Строится гистограмма интенсивностей пикселей.
- Предполагается бимодальное распределение (фон + объект).
- Алгоритм автоматически выбирает порог t .

Формально, порог t выбирается таким образом, чтобы максимизировать межклассовую дисперсию или минимизировать внутриклассовую дисперсию.

$$\sigma_b^2(t) = w_1(t)w_2(t)(\mu_1(t) - \mu_2(t))^2$$

Где:

- w_1, w_2 — доли классов (фона и объекта),
- μ_1, μ_2 — их средние интенсивности.

Эффект:

- Автоматический подбор одного глобального порога.
- Хорошо работает при равномерном освещении.
- Плохо работает при градиенте освещённости.

Адаптивная бинаризация

В отличие от глобальных методов, таких как Otsu-бинаризация, адаптивная бинаризация позволяет преобразовать полутоновое изображение в бинарное, используя локальный порог. Это особенно эффективно для изображений с неравномерным освещением, где один глобальный порог не может корректно разделить фон и объект.

Порог $T(x, y)$ вычисляется для каждого пикселя (x, y) в скользящем окне на основе окружающих пикселей:

$$T(x, y) = \text{mean/gaussian in window} - C$$

Где:

- **mean/gaussian in window** — это среднее или взвешенное среднее (с гауссовым распределением) значение интенсивности пикселей в локальном окне вокруг (x, y) .
- **C** — это константа, которая тонко настраивает порог, позволяя регулировать чувствительность к границам.

Варианты:

- **Adaptive Mean:** Использует среднее значение интенсивности в окне.
- **Adaptive Gaussian:** Использует взвешенное среднее с гауссовым распределением в окне, что придает больший вес центральным пикселям.

Эффект:

- Корректно разделяет фон и объект даже при сильном градиенте освещённости.
- Идеально подходит для обработки документов, сканов и текстов, где качество печати или освещения может быть переменным.

Бинаризация



При корректно выбранном пороге объекты чётко отделяются от фона. При неудачном пороге теряются детали, появляются шумовые области или происходит слияние объекта с фоном:

- усиливается структурная форма объектов;
- устраняются полутоновые переходы;
- повышается контраст между классами;
- существенно упрощается дальнейший анализ.

Предварительная обработка для бинаризации

Бинаризация сама по себе редко работает хорошо. Для улучшения её эффективности перед бинаризацией часто выполняется морфологическое сглаживание, чтобы сделать объект более однородным и улучшить разделимость классов.



01

Перевод в градации серого

Исходное изображение переводится из цветного в градации серого.



02

Морфологическая операция Closing

Применяется операция **closing** с квадратным ядром 7×7 , которая сглаживает текстуру, заполняет мелкие тёмные разрывы и делает области более однородными.



03

Otsu-бинаризация

После сглаживания выполняется Otsu-бинаризация для автоматического определения оптимального порога.



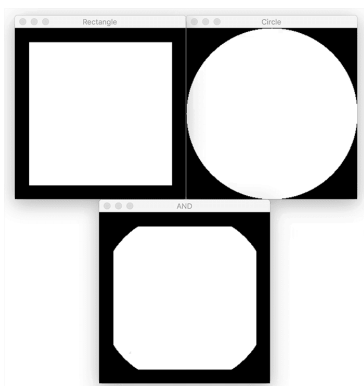
04

Получение маски

В результате этих шагов получается чистая чёрно-белая маска.

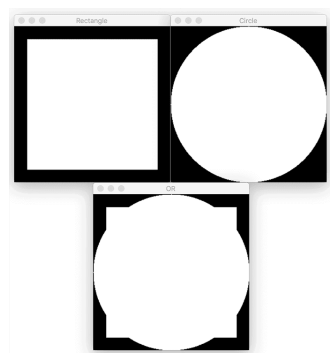
Логические (побитовые) операции над бинарными масками

Поскольку бинарные маски содержат только значения 0 и 1 (или 0 и 255), к ним можно применять логические (побитовые) операции. Эти операции позволяют эффективно комбинировать и уточнять области интереса, выделяя пересечения, объединяя сегменты или формируя сложные регионы из простых масок.



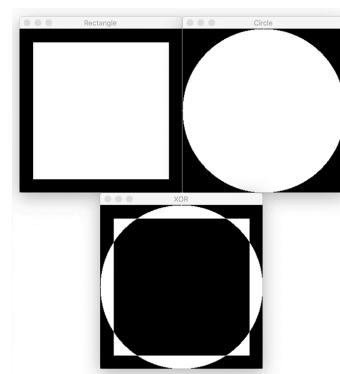
AND — пересечение

Оставляет только те пиксели, которые равны 1 (или 255) в обеих масках, выделяя общие области.



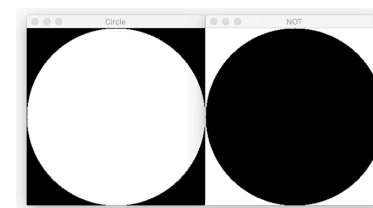
OR — объединение

Пиксель становится 1 (или 255), если он равен 1 хотя бы в одной из масок, сливая области.



XOR — исключаящее ИЛИ

Оставляет области, принадлежащие только одной из масок, игнорируя общие и пустые части.



NOT — инверсия

Меняет значения пикселей на противоположные (0 становится 1, 1 становится 0), эффективно меняя объект и фон местами.

Применение маски



+



Поиск контуров на бинарной маске

После бинаризации изображение состоит из двух областей — объект и фон. Это позволяет применять алгоритм `findContours`, который извлекает границы связанных компонент.

Что происходит:

- Из бинарного изображения извлекаются все контуры (`cv2.findContours`).
- Используется режим `RETR_EXTERNAL` — берём только внешние границы.
- Контуры сортируются по площади.
- Выбираются несколько самых крупных (например, 5).

Эффект:

- Каждая белая область превращается в набор точек, описывающих её границу;
- Можно вычислять площадь, периметр, аппроксимировать форму;
- Можно отфильтровать шум, оставив только крупные объекты.

Поиск контуров на бинарной маске



Упрощение и аппроксимация контуров

После извлечения контуров их часто необходимо упростить или аппроксимировать более простыми геометрическими фигурами. Это позволяет уменьшить вычислительную сложность, отфильтровать шум и получить компактное описание формы объекта, что важно для дальнейшего анализа и классификации.

1

Полигональная аппроксимация

Упрощает контур до набора вершин, формируя многоугольник. Параметр `epsilon` регулирует степень упрощения: чем он больше, тем грубее аппроксимация, но тем меньше вершин.

2

Выпуклая оболочка

Строит минимальную выпуклую фигуру, полностью охватывающую объект. Используется для анализа общей формы, выявления впадин и дефектов объекта.

3

Ограничивающие прямоугольники

Позволяют компактно описать положение и размер объекта. Различают осевой прямоугольник (`cv.boundingRect`) и минимальный по площади прямоугольник с поворотом (`cv.minAreaRect`).

4

Минимальная описывающая окружность

Находит наименьшую окружность, полностью содержащую контур. Полезна для быстрой оценки масштаба и общей компактности объекта.

5

Аппроксимация эллипсом

Подбирает эллипс, наиболее точно описывающий форму контура. Особенно эффективна для вытянутых, овальных или асимметричных объектов.

6

Аппроксимация прямой

Находит прямую, которая наилучшим образом соответствует набору точек контура. Используется для определения основного направления или ориентации объекта.

Упрощение и аппроксимация контуров



Гомография

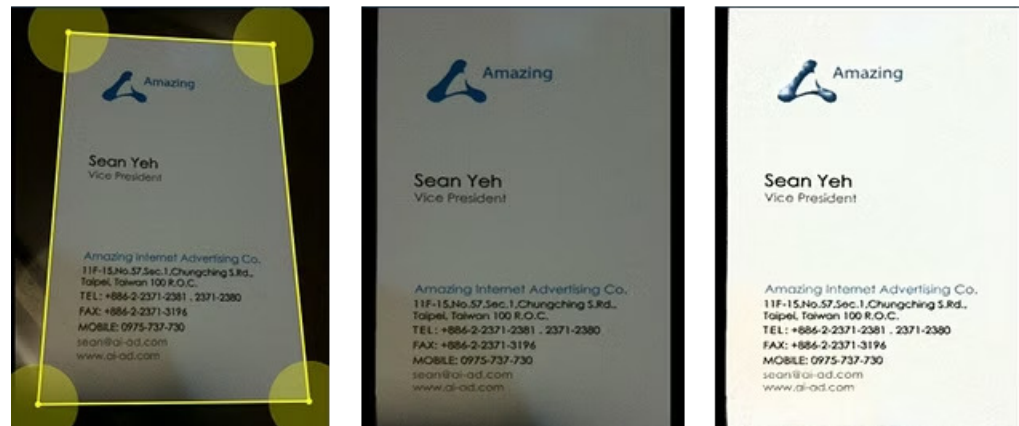
Гомография — это проективное преобразование, описывающее отображение одной плоскости в другую.

Если объект лежит в одной плоскости (например, номерной знак), то его изображение при перспективной съёмке связано с фронтальным видом матрицей 3×3 :

$$x' \sim Hx$$

Где:

- $x = (x, y, 1)^T$ — точка в исходном изображении,
- H — матрица гомографии,
- x' — точка в новом (выпрямленном) изображении.



Применение гомографии для выпрямления номерных знаков

Номерной знак представляет собой плоский объект. Если камера зафиксировала его под углом, то на изображении знак искажается до трапеции. Гомография позволяет устранить эти перспективные искажения.

